# Video 9 – The Linear Model II

## Video Contents



## Video Synopsis

**Introduction** **04:32 – 07:26**

The lecture is started by summarising what we already know about linear models. Linear classification and linear regression via the pseudo-inverse are fully understood (see lecture 3) and their VC dimension can be calculated explicitly as d+1 with d the number of features. In lecture 3 we touched upon non-linear transforms and we showed that they are potentially powerful ways of expanding linear models for use with more complex data sets showing non-linear trends or patterns. However, there are also dangers in using this approach related to generalization which will be considered in this and following lectures. This lecture will first look at non-linear transformations and will then discuss logistic regression as a new linear classification algorithm.

**Non-linear Transforms** ***07:26 – 10:12***

Non-linear transformations are formalised as a transform from the features in the x-space to a new z-space:

$$\boldsymbol{z} = \Phi(\boldsymbol{x})$$

The idea is that, by allowing the features to follow predefined (by the transformation) non-linear trends, a linear model can be used to successfully model the non-linear trend / pattern. The model for a linear classifier using transformed features can be described as:

$$\mathbf{y}=\mathbf{sign}(\tilde{\boldsymbol{w}}^T\Phi(\boldsymbol{x}))$$

and the same model for linear regression using transformed features would be:

$$\boldsymbol{y} = \tilde{\boldsymbol{w}}^T\Phi(\boldsymbol{x})$$

The 'squiggly' w indicates that this is the weight vector applied to the transformed features.

**The price of non-linear transforms** **10:12 – 13:02**

We know that for a linear model with d features, the VC dimension is d+1. Suppose you decide on a non-linear transformation that results in $\tilde{d} = 3d$ features, then the VC dimension is suddenly far higher (a factor $\frac{3d+1}{d+1}$). As we know from the lectures on the theory of generalization, this has a direct impact on the amount of data that will be required to train a model and still have generalizable results. Note that the VC dimension of the transformed model is at most, rather than exactly $\tilde{d} + 1$ as it is possible that not all non-linearly transformed features result in valid, distinguishable data points in the original x-space. For example if you have the feature vector x = [x1,x2] (VC dimension 3), a non-linear transform to [x1,x2,x1x2,x1,x2] would not result in a VC dimension of 7 as the last two-features in the non-linearly transformed data are simple repeated features. This is a rather obvious example, but this may also happen in less obvious ways. In general the VC dimension will be close to $\tilde{d} + 1$.

**Non-linear data transformation and data snooping** ***13:02 – 24:02***

Two extreme cases where non-linear transformation could be used, are discussed. One barely non-linear due to a few outliers and the other clearly non-linear. In the case of the barely non-linear data it becomes clear immediately that trying to perform a non-linear transformation is not a good idea. This non-linear transformation would likely improve performance in sample, but at least at the expense of requiring far more data to ensure generalization of the results due to the higher VC dimension of the new model with non-linear features. In such cases, it is highly likely that you are modelling noise rather than real non-linear trends.

The second case, in which the class of the data seems determined by their distance from the origin, is begging for a second order transformation as with a second order transformation you can describe a circle around the origin that separates data points from one class from the others. Hence, a first step could be:

$$\boldsymbol{z} = (1, x_1, x_2, x_1x_2, x_1^2, x_2^2)$$

The issue now is that the VC dimension has gone up from 3 (for $\boldsymbol{x} = (1, x_1, x_2)$) to 8! As this would cost you a lot of extra data (which potentially you don't have) you may be tempted to optimise the transformation. Eventually, this would lead to a one-feature non-linear transformation that describes the distance from the origin:

$$\boldsymbol{z} = (x_1^2 + x_2^2 - 0.6)$$

With a VC dimension of 1. This sounds like a good result (even better than the original!) but the problem is that you have looked at the data to decide on this transformation. So in effect, you have been part of the learning algorithm and you have already gone through a number of hypotheses before arriving at this transformation; you have come to the conclusion that the best transformation corresponds to a weight vector with certain elements set to 0 and have discarded these from the weight vector (and feature transformation) altogether.. This means that, rather than considering the VC dimension of the final transformation, you should consider the VC dimension of the original transformation.

This part of the lecture concludes by defining this type of use of the data to choose a sub-set of hypotheses as 'data snooping'. There are other, less obvious, ways that you can fall in the trap of data snooping and these will be discussed in later lectures. Data snooping is not necessarily bad, but you will have to account for the fact that you have been part of the training process! This can be done in several ways and we will look at this in future lectures.

**Logistic Regression – The Model** **24:02 – 37:58**

Logistic regression is introduced as a further linear model in which the sign function that we have previously used in linear classification, is replaced by the logistic function which allows us to interpret the output of the model as a probability. This is possible as the output of the logistic function:

$$\theta(s) = \frac{e^s}{1 + e^s}$$

where s is the signal that we have seen previously for linear models:

$$s = \boldsymbol{w}^T\boldsymbol{x}$$

has an output between 0 and 1. There are other logistic functions that we could have chosen, but this one has very nice mathematical properties.

We look at the example of predicting heart attacks based on, for example, 10 personal, medical and lifestyle features. With these features, we could not conceivably predict correctly whether they will result in a heart attack in every single case. It is more realistic to predict the probability of heart attack instead. However, interestingly the data that we would train this model with, do not provide probabilities as the target value to learn, but actual occurrences (or not) of heart attacks. In other words, with every example of our feature vector **x**, we will have a y = +1 or -1 to indicate that this particular combination of features resulted in a heart attack or not.

So what the model will need to learn based on examples is the probability that a certain example **x** would lead to a heart attack or not:

$$P(y|\boldsymbol{x}) = \begin{cases} f(x) & \text{for y=+1} \\ 1 - f(x) & \text{for y=-1} \end{cases}$$

In other words, we want our final hypothesis g(**x**) to represent the true underlying probability of heart attack f(**x**) using the logistic regression model: $\theta(\boldsymbol{w}^T\boldsymbol{x})$. Unlike for linear regression and linear classification, finding the weight vector **w** that provides us with the best fit does not have a one-step solution. Before we can discuss what algorithm can be used to find a solution, we need to define a new error function.

**Logistic Regression – The Error Measure** **37:58 – 51:24**

The error measure is introduced as a pragmatically chosen, but plausible error measure based on likelihood. Suppose you choose a hypothesis h, you could then assess on all training data points how likely it is that h(**x**) resulted in the events (heart attack or not) as described by your training data. If you conclude that the training data is plausible (i.e. many of the heart attacks are predicted by h given the data **x**), then you can conclude that h is a good candidate. If the data is not plausible under a given h, then you need to look further. Such a hypothesis can be mathematically described as:

$$P(y|\boldsymbol{x}) = \begin{cases} \theta(\boldsymbol{w}^T\boldsymbol{x}) & \text{for y=+1} \\ 1 - \theta(\boldsymbol{w}^T\boldsymbol{x}) & \text{for y=-1} \end{cases}$$

As for the logistic function the following holds: $\theta(-s) = 1 - \theta(s)$ we can rewrite this as:

$$P(y|\boldsymbol{x}) = \begin{cases} \theta(\boldsymbol{w}^T\boldsymbol{x}) & \text{for y=+1} \\ \theta(-\boldsymbol{w}^T\boldsymbol{x}) & \text{for y=-1} \end{cases}$$

By observing that the two cases are for y = +1 and -1, we can rewrite the cases as:

$$P(y|\boldsymbol{x}) = \begin{cases} \theta(y\boldsymbol{w}^T\boldsymbol{x}) & \text{for y=+1} \\ \theta(y\boldsymbol{w}^T\boldsymbol{x}) & \text{for y=-1} \end{cases}$$

And as both cases are the same, there is no longer a need for special cases for both values of y:

$$P(y|\boldsymbol{x}) = \theta(y\boldsymbol{w}^T\boldsymbol{x})$$

This equation models the likelihood of **w** (which is effectively our hypothesis) given the input **x** and the output y. As we will want to find a likelihood that covers all data points, we want to extend this to the full data set. We thus look for the likelihood that the first x results in the first y given the weight vectors and the second x results in the second y given the weight vector and … the n$^{th}$ x results in the n$^{th}$ y given the weight vector:

$$P(y_1|\boldsymbol{x_1}) \text{ and } P(y_2|\boldsymbol{x_2}) \text{and } \ldots \text{ and } P(y_N|\boldsymbol{x_N})$$

Which can be calculated as a product of the individual probabilities:

$$P(y_1|\boldsymbol{x_1})P(y_2|\boldsymbol{x_2}) \ldots P(y_N|\boldsymbol{x_N}) = \prod_{n=1}^{N} P(y_n|\boldsymbol{x_n}) = \prod_{n=1}^{N} \theta(y_n\boldsymbol{w}^T\boldsymbol{x}_n)$$

Unlike the error measures that we have seen thus far, we would like to maximise this function with respect to **w** as we would like to maximise the likelihood across all examples that the weight vector correctly predicts the occurrence of a heart attack. We will now show that maximising this

function is the same as minimising a related function. Maximising a function is the same as maximising the log of a function as long as the function itself is always > 0 (which it is as the logistic function is > 0) and because the log is a monotonically increasing function:

$$\underset{\boldsymbol{w}}{\operatorname{argmax}} \prod_{n=1}^{N} \theta(y_n \boldsymbol{w}^T \boldsymbol{x}_n) \rightarrow \underset{\boldsymbol{w}}{\operatorname{argmax}}\, ln\left(\prod_{n=1}^{N} \theta(y_n \boldsymbol{w}^T \boldsymbol{x}_n)\right)$$

We can further scale by the number of training samples N:

$$\underset{\boldsymbol{w}}{\operatorname{argmax}}\, ln\left(\prod_{n=1}^{N} \theta(y_n \boldsymbol{w}^T \boldsymbol{x}_n)\right) \rightarrow \underset{\boldsymbol{w}}{\operatorname{argmax}} \frac{1}{N} ln\left(\prod_{n=1}^{N} \theta(y_n \boldsymbol{w}^T \boldsymbol{x}_n)\right)$$

Furthermore, we can introduce a minus sign to change from maximization to minimization:

$$\underset{\boldsymbol{w}}{\operatorname{argmax}} \frac{1}{N} ln\left(\prod_{n=1}^{N} \theta(y_n \boldsymbol{w}^T \boldsymbol{x}_n)\right) \rightarrow \underset{\boldsymbol{w}}{\operatorname{argmin}} -\frac{1}{N} ln\left(\prod_{n=1}^{N} \theta(y_n \boldsymbol{w}^T \boldsymbol{x}_n)\right)$$

And because $\ln(x \cdot y) = \ln(x)+\ln(y)$ and $-\ln(y) = \ln\left(\frac{1}{y}\right)$ we can rewrite this as:

$$\underset{\boldsymbol{w}}{\operatorname{argmin}} \frac{1}{N} \sum_{n=1}^{N} ln\left(\frac{1}{\theta(y_n \boldsymbol{w}^T \boldsymbol{x}_n)}\right)$$

We can now substitute the logistic function after rewriting it slightly:

$$\theta(s) = \frac{e^s}{1+e^s} = \frac{1}{\frac{1}{e^s}+1} = \frac{1}{1+e^{-s}}$$

$$\underset{\boldsymbol{w}}{\operatorname{argmin}} \frac{1}{N} \sum_{n=1}^{N} ln\left(\frac{1}{\frac{1}{1+e^{-y_n \boldsymbol{w}^T \boldsymbol{x}_n}}}\right) = \underset{\boldsymbol{w}}{\operatorname{argmin}} \frac{1}{N} \sum_{n=1}^{N} ln\left(1+e^{-y_n \boldsymbol{w}^T \boldsymbol{x}_n}\right)$$

which will act as the in-sample error for logistic regression:

$$E_{in}(\boldsymbol{w}) = \frac{1}{N} \sum_{n=1}^{N} ln\left(1+e^{-y_n \boldsymbol{w}^T \boldsymbol{x}_n}\right)$$

Which looks very similar to previous error measures with $e(h(\boldsymbol{x}_n), y_n) = ln\left(1+e^{-y_n \boldsymbol{w}^T \boldsymbol{x}_n}\right)$ the point wise error measure. This is an intuitively appealing error measure: imagine that $\boldsymbol{w}^T \boldsymbol{x}$ yields a positive number for an example with y=-1. In that case the point wise error will be the natural logarithm of a positive exponential function, which will give a large error. The same holds if $\boldsymbol{w}^T \boldsymbol{x}$ yields a negative number for an example with y=+1. If $\boldsymbol{w}^T \boldsymbol{x}$ and y have the same sign (i.e., the weight vector results in a correct prediction) then the error function will be the natural logarithm of 1 plus an exponential function with negative exponent, which will result in a very small number. Hence, intuitively the behaviour of this error function corresponds with what we expect from error functions.

This error function is called the cross-entropy error.

**Logistic Regression – The Learning Algorithm** **51:24 – 1:04:38**

Comparing the cross-entropy error function to the mean squared error function used in linear regression, shows that we cannot find a one-step solution. Instead, we will resort to an iterative solution that is used in many other machine learning algorithms as well: gradient decent. In general, you will only have local information on how your error behaves in response to varying the

weight vector w and in general the response of the error function to the weight vector can be very unpredictable and lumpy. Hence, the best you can do to find a minimum, is to assess in your current location (in terms of weight vector) in what direction the error function reduces most. In other words, how should you change your weight vector to reduce the error function most. This can be written mathematically as:

$$\Delta E_{in} = E_{in}(\boldsymbol{w}(0) + \eta\hat{\mathrm{v}}) - E_{in}\big(\boldsymbol{w}(0)\big)$$

The right-hand side of this equation corresponds to the value of the in-sample error at a point **w**(0) plus a small delta minus the value of the in-sample error at **w**(0). The delta in this equation $(\eta\widehat{\mathrm{v}})$ is by definition a scalar $\eta$ (Greek letter eta) multiplied by a unit vector (a vector with length 1) $\hat{\mathrm{v}}$. This is the same as the first order approximation of the derivative of a function multiplied by the delta with higher order terms ignored:

$$= \nabla E_{in}\big(\boldsymbol{w}(0)\big)^T \eta\hat{\mathrm{v}}$$

The symbol ∇ (nabla) in this equation indicates the gradient, which is the element-by-element derivative of a vector. Note that the delta in this case is η multiplied by a small change in the weight vector $\hat{\mathrm{v}}$, so we can rewrite as:

$$= \eta\nabla E_{in}\big(\boldsymbol{w}(0)\big)^T \hat{\mathrm{v}}$$

As our goal is to minimise the error function, we want to choose $\hat{\mathrm{v}}$ such that we make each step as big as possible in the negative direction. As the gradient points in the direction of steepest ascent, we will find the steepest descent (i.e. the direction that will yield the biggest decrease in error) in the opposite direction. As $\hat{\mathrm{v}}$ should be a unit vector, we do need to normalise by the length of the vector **w**(0):

$$\hat{\mathrm{v}} = -\frac{\nabla E_{in}\big(\boldsymbol{w}(0)\big)}{\big\|\nabla E_{in}\big(\boldsymbol{w}(0)\big)\big\|}$$

At the end the influence of the value of $\eta$ is investigated. $\eta$ is included to make sure the steps taken are not too big that they violate our assumption of (local) linearity. However, too small an $\eta$ will result in very slow meaningful updates. To solve this, we can automatically scale the learning rate to the current slope encountered, by multiplying it by the current slope. This happens to be exactly $\big\|\nabla E_{in}\big(\boldsymbol{w}(0)\big)\big\|$. Hence a suitable update in gradient descent is:

$$\Delta\boldsymbol{w} = -\eta\nabla E_{in}\big(\boldsymbol{w}(0)\big)$$

**Summary of Logistic Regression Algorithm** **1:03:50 - 1:04:30**

The logistic regression algorithm can be summarised as:
Initialise the weights at t(0) to **w**(0)
for t=0,1,2,… do
    Compute the gradient
    Update the weights: $\boldsymbol{w}(t+1) = \boldsymbol{w}(t) - \eta\nabla E_{in}$
    Repeat until error sufficiently low
Return the final weights

**Summary of Linear Models** **1:04:30 - 1:06:45**

The lecture concludes with an overview of linear models:

- We started with Perceptrons that allow us to classify. They used classification error and algorithms such as the PLA or the Pocket algorithm.
- We then saw linear regression that allows us to approximate a real value. Linear regression used the mean squared error and we saw that there is a one-step solution using the pseudo-inverse. In practice you will often use iterative methods because the tend to be more stable.

- Finally, we saw logistic regression that allows us to determine a probability of something happening. It uses cross-entropy as its error function and gradient descent during training.